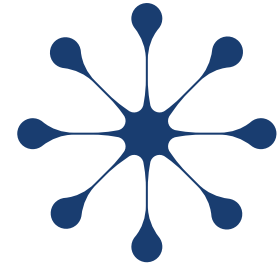




UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
Universidad del Perú, Decana de América



TECLADO MATRICIAL PARA COMANDOS AT

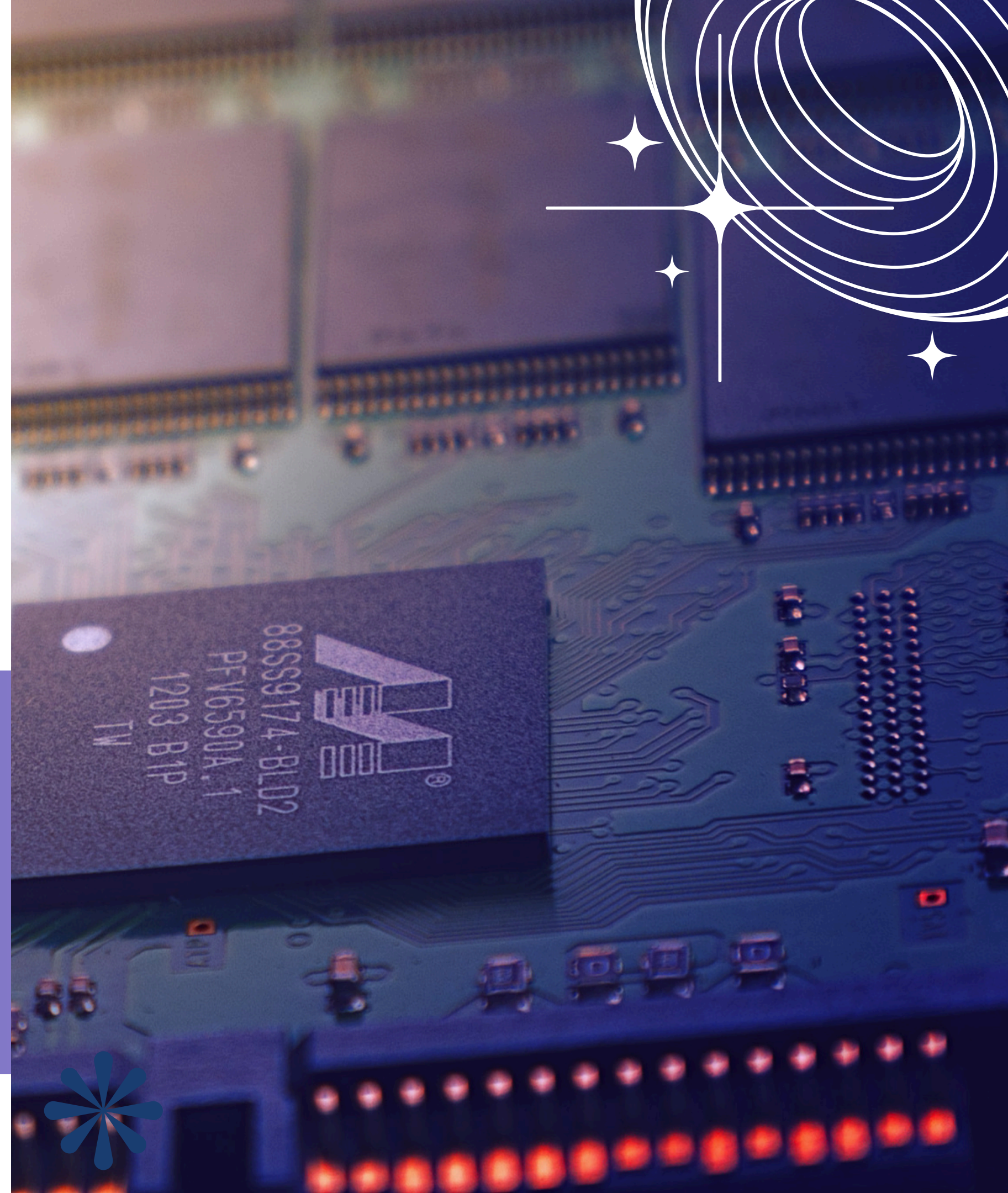
Integrantes:

Gomez Prada, Alonso Abraham

Silva Centeno, Bryan Rodolfo

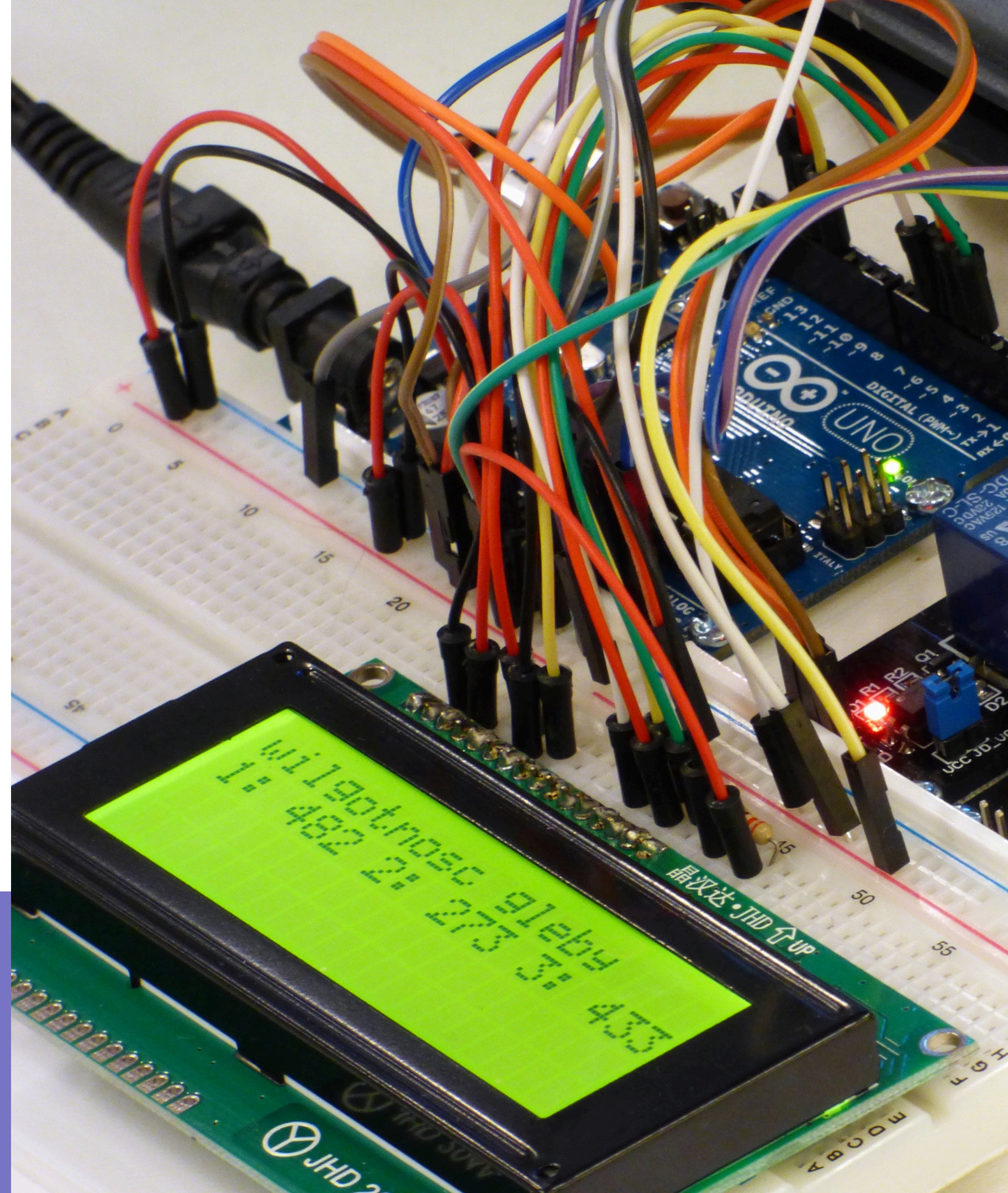
Huiman Vargas, Juan Jesus

Zamora de la Cruz, Pablo Alonso



DESCRIPCIÓN DEL SISTEMA

El emulador AT implementado en Arduino Uno R3 permite la recepción, validación, procesamiento y transmisión de comandos AT. La interacción se realiza mediante un teclado matricial y una pantalla LCD, mientras que una Máquina de Estados Finita coordina cada etapa del funcionamiento. Además, incorpora un módulo SIM800L GSM/GPRS, que amplía las capacidades del sistema al permitir la comunicación a través de la red celular mediante comandos AT y llamadas de voz. Para facilitar el monitoreo del proceso, el sistema utiliza indicadores LED que representan el estado operativo actual.



- ✓ *Arquitectura modular y escalable.*
- ✓ *Bajo consumo de recursos del Arduino.*
- ★ ✓ *Interfaz intuitiva mediante teclado y OLED.*
- ✓ *Integración con comunicación celular real.*

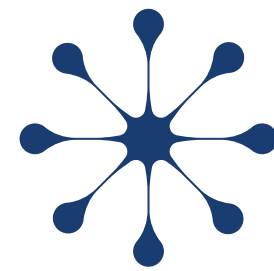
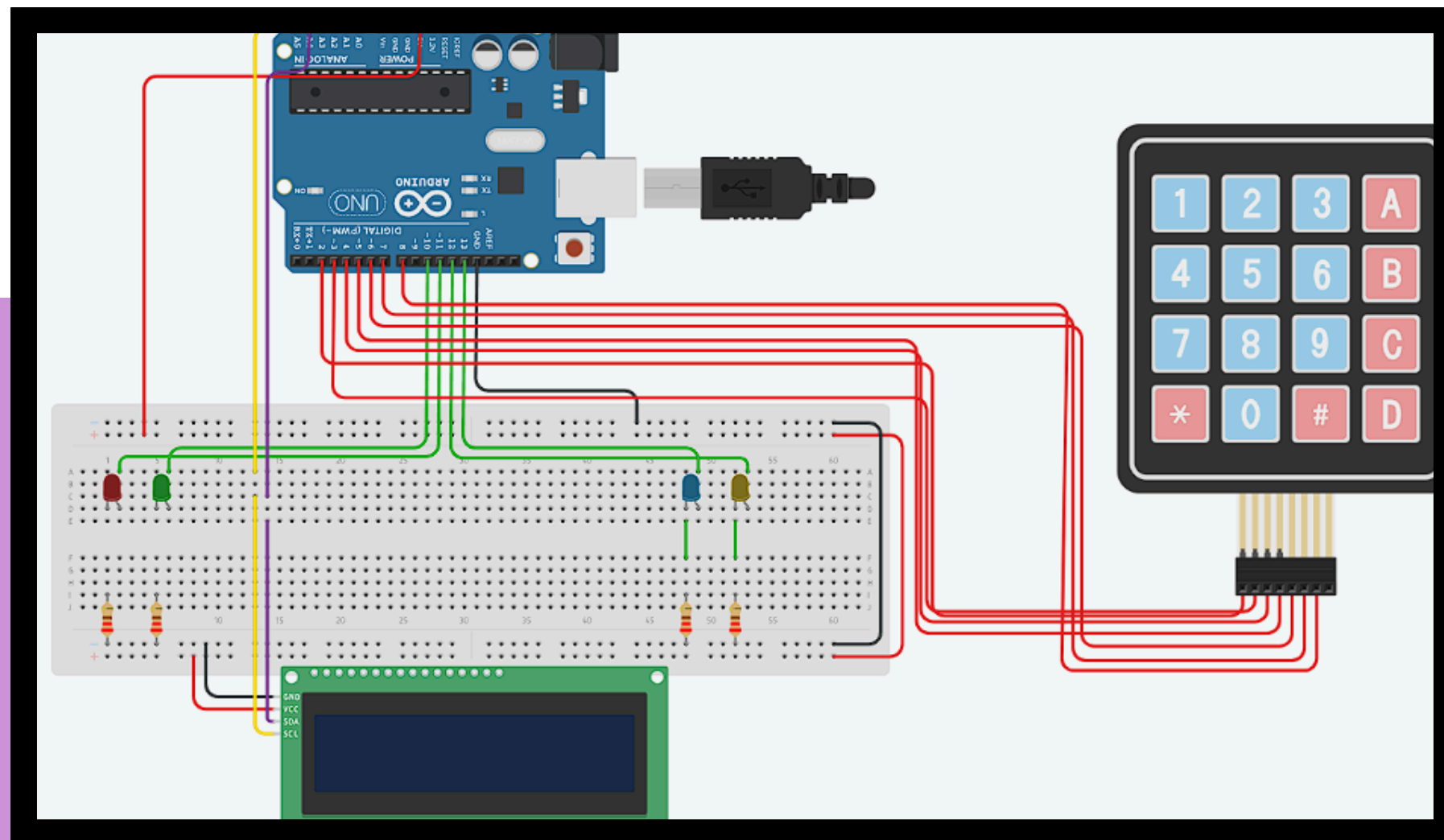


DIAGRAMA DE CONEXIONES



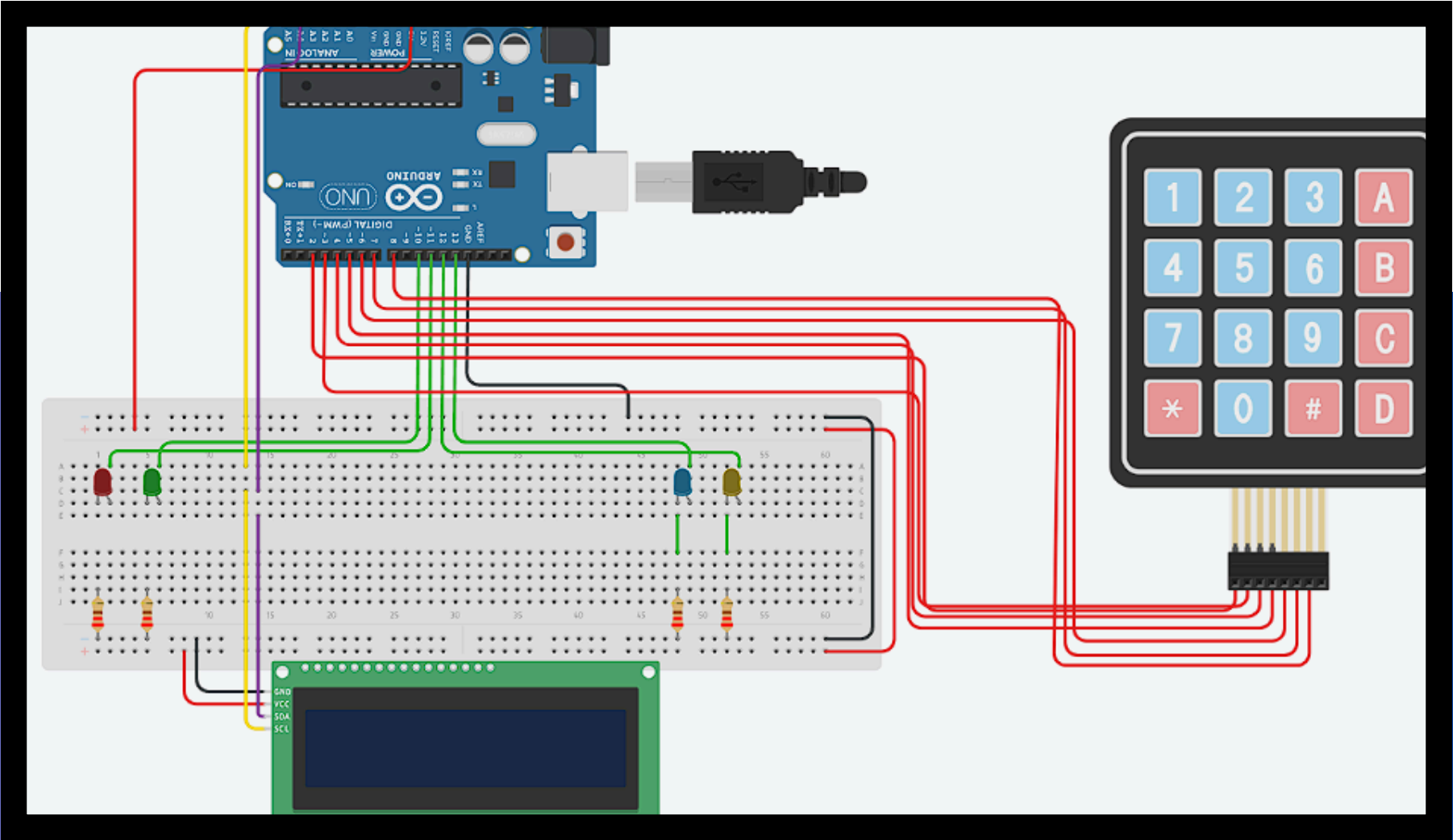
Componente Físico	Pin del componente	Pin en Arduino Uno	Función / Tipo de Señal
Pantalla LCD	VCC	5 V	Alimentación (5 V DC)
	GND	GND	Referencia de Tierra
	SCL	A5	Reloj del Bus I2C (Serial Clock)
	SDA	A4	Datos del Bus I2C (Serial Data)

- *Comunicación mediante protocolo I²C.*
- *Utiliza únicamente los pines A4 (SDA) y A5 (SCL).*
- *Resolución de 128×64 píxeles.*
- *Muestra comandos, estados y mensajes de error.*
- *Reduce el uso de pines respecto a una pantalla LCD convencional.*

Componente Físico	Pin del componente	Pin en Arduino Uno	Función / Tipo de Señal
Teclado Matricial 4x4	Fila 1	4	Salida Digital (Barrido FSM)
	Fila 2	5	Salida Digital (Barrido FSM)
	Fila 3	6	Salida Digital (Barrido FSM)
	Fila 4	7	Salida Digital (Barrido FSM)
	Columna 1	8	Entrada Digital (Internal Pull-Up)
	Columna 2	9	Entrada Digital (Internal Pull-Up)
	Columna 3	10	Entrada Digital (Internal Pull-Up)
	Columna 4	11	Entrada Digital (Internal Pull-Up)



DIAGRAMA DE CONEXIONES



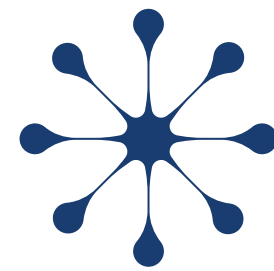
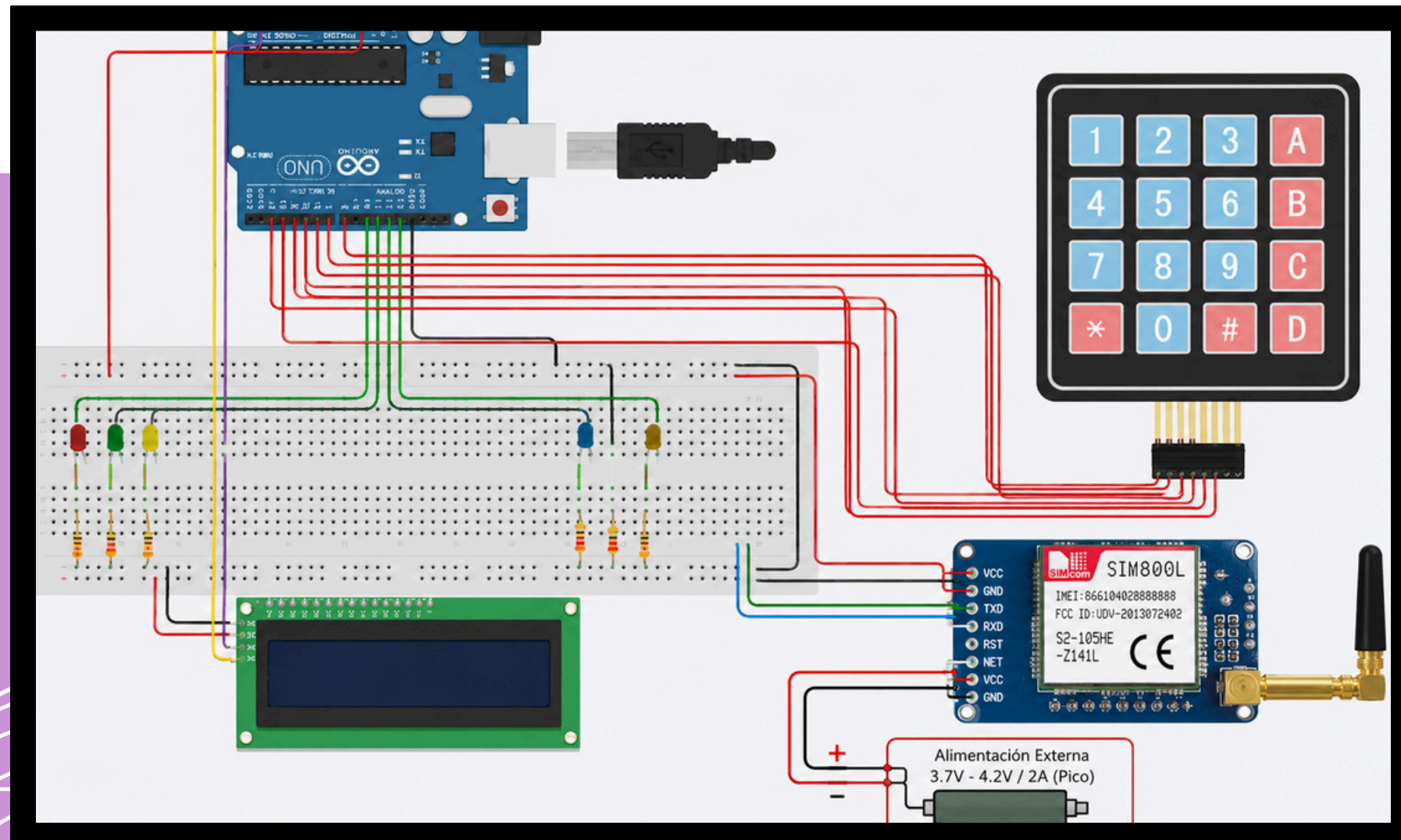


DIAGRAMA DE CONEXIONES



SIM800L	Arduino
VCC	Fuente 4V externa
GND	GND
TX	RX (3)
RX	TX (2)

- 📶 *SIM800L GSM/GPRS: permite realizar llamadas y enviar comandos AT mediante comunicación UART.*
- ☎️ *ATD: inicia una llamada al número ingresado.*
- 📴 *ATH: finaliza una llamada activa.*
- 📶 *Red GSM: proporciona conectividad celular para la transmisión de voz y datos.*
- ↔️ *UART 9600 bps: canal de comunicación entre el Arduino y el módulo SIM800L.*



CÓDIGO EN ARDUINO

1- Estructura FSM

```
void loop() {  
  char teclaPresionada = escanearTeclado();  
  
  switch (estadoActual) {  
    case IDLE:      /* Monitorea reposo y cambia LEDs V/A */ break;  
    case RECEIVE:   /* Clasifica si es *, # o datos */      break;  
    case STORE:     /* Carga macros o guarda en RAM */     break;  
    case SEND:      /* Transmite "AT+" + buffer por UART */ break;  
    case CLR:       /* Purgado físico de memoria RAM */     break;  
    case ERROR_OLED: /* Alarma visual y manejo de Overflow */ break;  
  }  
}
```

Implementa un bucle de control basado en una Máquina de Estados Finita (FSM). Al prescindir de retardos lineales, el microcontrolador optimiza el tiempo de CPU y procesa las transiciones lógicas entre la edición, el borrado y la transmisión en microsegundos, garantizando la ejecución fluida del sistema.

2- Filtro de seguridad

```
// Validación del límite físico del búfer (Máximo 4 caracteres)  
if (indiceBuffer < 4) {  
  bufferComando[indiceBuffer] = teclaPresionada;  
  indiceBuffer++;  
  bufferComando[indiceBuffer] = '\0';  
  Serial.println(teclaPresionada); // Telemetría serial  
} else {  
  estadoActual = ERROR_OLED; // Excepción por Overflow  
}
```

Protección de fronteras por software aplicada directamente sobre el búfer estático en la memoria RAM. El algoritmo intercepta en tiempo real cualquier intento de ingresar un quinto dígito, desviando el flujo de ejecución hacia un caso de manejo de excepciones (Overflow).



CÓDIGO EN ARDUINO

3- La salida de datos

```
// Envío del comando estructurado hacia el puerto serial
Serial.print("AT+");
Serial.println(bufferComando);
```

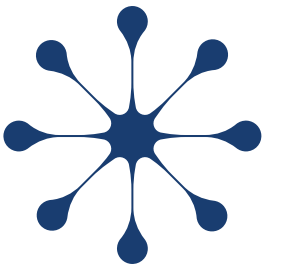
Módulo encargado del formateo y transmisión de los comandos AT generados por el usuario. El sistema concatena la información almacenada en memoria y la envía a través del puerto serial UART a 9600 bps, garantizando una comunicación estable y sincronizada con los dispositivos externos.

4- Comunicación GSM

```
case '2':
    Serial.println("Macro: Comprobando calidad de
mySerial.println("AT+CSQ");
    break;
```

Subsistema responsable de la interacción con el módulo SIM800L mediante comandos AT. Permite consultar el estado de la red celular, verificar la intensidad de señal y gestionar servicios de comunicación GSM, ampliando las capacidades del sistema hacia aplicaciones reales de telecomunicaciones móviles.

REPORTE DE RECURSOS



Uso de Memoria Flash

Parámetro	Valor
Mem. Flash Utilizada	15020 Bytes
Mem. Flash Disponible	32256 Bytes
Porcentaje Utilizado	46%

Uso de Memoria Ram

Parámetro	Valor
Mem. Ram Utilizada	795 Bytes
Mem. Ram Disponible	2048 Bytes
Porcentaje Utilizado	38%

El perfil de memoria del Arduino UNO refleja un diseño altamente optimizado. El uso del 46% de la memoria Flash se debe principalmente a la inclusión de las librerías gráficas y los mapas de bits para las alarmas en el display. Por otro lado, al implementar un búfer estático, el consumo de memoria RAM se estabiliza en un 38%, mitigando riesgos de fragmentación y garantizando un margen libre del 62% para la futura integración de componentes extras.

MEDICIÓN DE MÉTRICAS

- *Métrica a medir: Latencia de Despacho HMI*

Para medir esta métrica, se realizará una auditoría temporal del búfer mediante Timestamps en el Monitor Serial del Arduino IDE (COM3).

$$T_{\text{transmisión}} = \frac{9 \text{ caracteres} \times 10 \text{ bits/carácter}}{9600 \text{ bps}} = 9.375 \text{ ms}$$

Tiempo que tarda el Arduino en enviar completamente el comando AT+1254 a través del puerto serial UART configurado a 9600 bps.

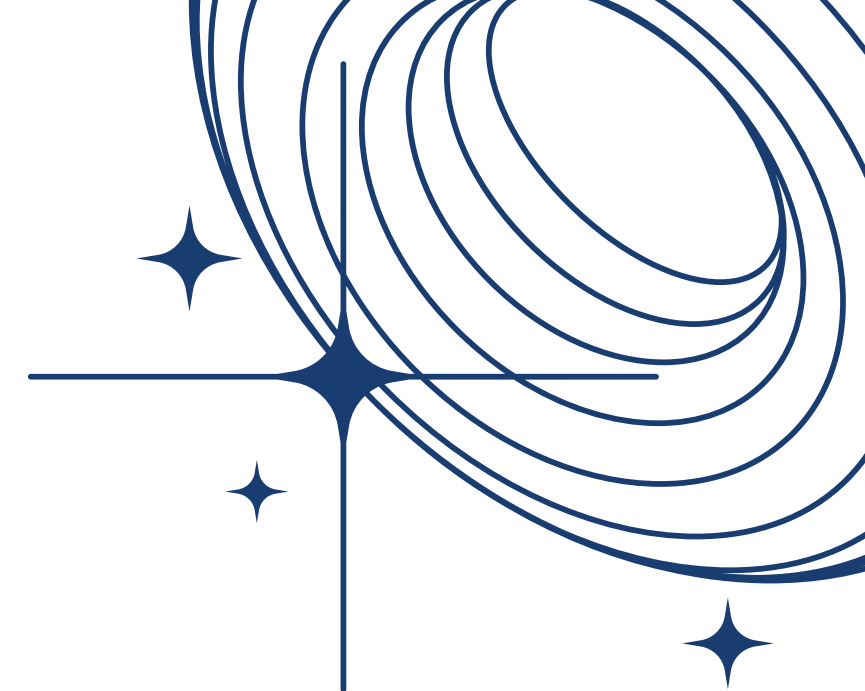
```
Output  Serial Monitor  X
Message (Enter to send message to 'Arduino UNO' on 'COM3')

15:17:09.010 -> 1
15:17:09.454 -> 2
15:17:09.869 -> 5
15:17:10.417 -> 4
15:17:11.599 -> AT+1254
```

$$\Delta T = 15 : 17 : 11.599 - 15 : 17 : 10.417 = 1.182 \text{ s}$$

Tiempo de reacción del operador. El sistema registra con precisión un intervalo de 1.182 segundos entre la última pulsación y la orden de transmisión (), validando la estabilidad de la FSM en modo de edición.*





¡Gracias!

